

Deep Learning for Jet Classification: a Data-Efficiency and Calibration Study on the 14 TeV Top-Tagging Benchmark

[Student Name]

BS Physics, Final-Year Project

Supervisor: Dr. Ram Chand

Department of Physics

Begum Nusrat Bhutto Women University, Sukkur

in collaboration with CERN

May 28, 2026

*This thesis is submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Physics at the Begum Nusrat Bhutto Women University, Sukkur.*

Abstract

This thesis presents a side-by-side study of four jet-classification architectures — a gradient-boosted decision tree (BDT) on engineered jet observables, a convolutional neural network (CNN) on 40×40 jet images, a graph network (ParticleNet) and a self-attention model (Particle Transformer) — on the publicly available Top-Quark Tagging Reference Dataset of Kasieczka *et al.* (Zenodo 2603256). Beyond reproducing the headline ROC comparison, two practically relevant questions are addressed. First, the *data efficiency* of each model is quantified by re-training from scratch at four training-set sizes spanning a decade (5×10^3 to 5×10^4 jets) and measuring AUC and background rejection at fixed signal efficiency. Second, the *probability calibration* of each model is evaluated using reliability diagrams and the Expected Calibration Error, and a single-parameter temperature rescaling is fitted on the validation set as a simple corrective. The BDT is found to be markedly data-efficient at the expense of absolute discriminating power, while the graph and attention models keep improving with N_{train} — the Particle Transformer in particular climbs from the weakest model at 5×10^3 jets to the strongest at 5×10^4 . Ranking and calibration prove to be decoupled: the jet-image CNN is by far the worst-calibrated model ($\text{ECE} \approx 0.08$) despite a mid-pack ranking, whereas the boosted tree and graph network are already well-calibrated ($\text{ECE} \approx 0.007$). All results are reproducible on a single CPU laptop in about an hour (minutes on a GPU) using the open-source pipeline released with this thesis.

Acknowledgements

I thank Dr. Ram Chand for proposing this project and for his patient guidance, the Department of Physics at Begum Nusrat Bhutto Women University for providing access to compute and library resources, and the maintainers of the CERN Open Data and Zenodo infrastructure for making the Top-Quark Tagging Reference Dataset publicly available. This thesis would not exist without the open ecosystem of NumPy, PyTorch, XGBoost, scikit-learn and matplotlib.

Contents

Abstract	2
Acknowledgements	3
1 Introduction	8
1.1 Context	8
1.2 Motivation	8
1.3 Aims and contributions	9
1.4 Thesis outline	9
2 Background	10
2.1 The Standard Model in one page	10
2.2 QCD and jet formation	10
2.3 The top quark	10
2.4 Machine-learning preliminaries	11
2.5 Evaluation metrics	11
3 Dataset and preprocessing	12
3.1 The Top-Quark Tagging Reference Dataset	12
3.2 Subset selection	12
3.3 Per-particle features	12
3.4 Jet-level features for the BDT	13
3.5 Jet images	13
3.6 Pairwise features for the Transformer	14
4 Models	15
4.1 BDT baseline	15
4.2 Jet CNN	15
4.3 ParticleNet	15
4.4 Particle Transformer	16
4.5 Training protocol	16

5	Headline results	17
5.1	Performance on the 50k subset	17
5.2	Score distributions and confusion matrices	18
5.3	Training dynamics	18
5.4	Interpretability of the BDT	18
5.5	Interpretability of the Particle Transformer	19
6	Data efficiency and calibration	22
6.1	Learning curves	22
6.2	Probability calibration	23
6.3	Summary table of calibration	24
7	Discussion	25
7.1	What this study did not do	25
7.2	Implications for analysis use	25
7.3	Future work	25
8	Conclusions	27
A	Reproducing the results	29
A.1	Environment	29
A.2	Dataset	29
A.3	Headline run	29
A.4	Research-angle study	29
A.5	Compiling the documents	29
B	Selected code excerpts	30
B.1	EdgeConv block (<code>src/particle_net.py</code>)	30
B.2	Pairwise attention bias (<code>src/particle_transformer.py</code>)	30

List of Figures

3.1	Distributions of the thirteen engineered jet-substructure features on the 50 k-jet test sample. Top jets in red, QCD jets in blue. Vertical dotted lines on the mass panel indicate the W and top masses.	13
3.2	Averaged p_T -weighted jet image for top jets (left), QCD jets (centre) and their pixel-wise difference (right). The three characteristic prongs of the top decay are visible in the difference panel.	14
5.1	ROC curves (left) and background rejection at fixed signal efficiency (right) for the four models, on 5×10^4 training and test jets each.	17
5.2	Classifier output distributions for the four models on the test split ($\log y$). Signal (top jets) in red, background (QCD jets) in blue.	18
5.3	Confusion matrices at the standard threshold of 0.5.	18
5.4	Training (dashed) and validation (solid) loss, validation AUC and learning rate schedule for the three neural models.	19
5.5	XGBoost gain-based feature importance for the BDT baseline.	19
5.6	Self-attention maps (head-averaged) of the Particle Transformer for two top and two QCD jets, across the three transformer layers.	20
5.7	Last-layer [CLS] attention weight as a function of constituent ΔR to the jet axis, for ten test jets each of the top and QCD classes.	21
6.1	Test AUC (left) and background rejection at $\varepsilon_S=0.5$ (right) as a function of the training-set size. Error bars on the AUC are 1σ from a 200-resample bootstrap.	22
6.2	Reliability diagrams (top) and score histograms (bottom) for the four models on the test set. Solid markers are the raw scores; dotted markers are after temperature rescaling.	23

List of Tables

5.1	Comparison of jet classification models on the Top Quark Tagging benchmark dataset.	17
6.1	Test-set AUC at $N_{\text{train}} = 5 \times 10^4$ and calibration metrics. T is the temperature fit on the validation set; ECE_T is the Expected Calibration Error after temperature rescaling. (Populated from <code>results/study_summary.json</code> by <code>tools/build_calibration_table.py</code> .)	24

Chapter 1

Introduction

1.1 Context

The Large Hadron Collider (LHC) at CERN accelerates protons up to 7 TeV per beam and brings them into collision 4×10^7 times a second. Each collision is recorded by a detector — ATLAS, CMS, ALICE or LHCb — as a multi-megabyte event consisting of energy deposits in calorimeters, hits in tracking systems and timing information. The Standard Model of particle physics predicts the statistical distribution of these events; departures from this prediction are how new physics would announce itself.

A central building block of the analysis chain is the *jet* — a collimated spray of hadrons that is the experimental shadow of a hard parton (quark or gluon) produced in the collision. Jets are reconstructed from the calorimetric and tracking information using sequential-recombination algorithms such as anti- k_T [6], and are subsequently classified by flavour or origin. The top quark is special: with a mass $m_t \simeq 172.76$ GeV and a width $\Gamma_t \sim 1.4$ GeV it decays before it can hadronise, almost always via $t \rightarrow Wb$. When produced with high transverse momentum, the entire $t \rightarrow Wb \rightarrow (q\bar{q})b$ decay chain ends up inside a single large-radius jet, which the experimentalist has to disentangle from the much more abundant background of QCD multijet events.

1.2 Motivation

Modern deep-learning architectures have closed most of the gap between human-engineered and machine-learned representations of jet substructure information. The standard reference benchmark for the field — the Top-Quark Tagging Reference Dataset [1] — is by now *saturated*: ParticleNet [2] achieved AUC $\simeq 0.985$ already in 2019, and the Particle Transformer [3] pushes this slightly further at the cost of substantially more compute.

What is rarely reported is the *cost* of those headline numbers: how much training data each architecture actually needs to reach its plateau, and how trustworthy its output

probabilities are when fed downstream into a likelihood fit or a binned analysis. Both questions are practically important. Monte-Carlo generation of the training samples is expensive — the public dataset contains 1.2×10^6 training jets, which took weeks of CPU time to produce. And the calibration of the score determines whether the tagger can be used as a probability or only as a ranking score.

1.3 Aims and contributions

This thesis sets out to:

1. Build a unified, reproducible single-machine pipeline that trains and evaluates four representative top-tagging architectures (BDT, CNN, ParticleNet, Particle Transformer) on the public reference dataset, with identical preprocessing and seeds.
2. Perform a *data-efficiency* sweep over four training-set sizes.
3. Perform a *probability-calibration* study using reliability diagrams and the Expected Calibration Error, before and after a single-parameter temperature rescaling.
4. Produce student-facing documentation (the companion tutorial), so the work can serve as teaching material for future BS-level projects at the host institution.

The pipeline runs end-to-end in under one hour on a CPU laptop.

1.4 Thesis outline

Chapter 2 reviews the necessary physics and machine-learning background. Chapter 3 describes the dataset and the preprocessing. Chapter 4 introduces each of the four models in detail. Chapter 5 presents the headline benchmark. Chapter 6 contains the data-efficiency and calibration study that forms the research contribution of the thesis. Chapter 7 discusses limitations and outlook, and Chapter 8 concludes.

Chapter 2

Background

2.1 The Standard Model in one page

The Standard Model is a renormalisable quantum field theory with gauge group $SU(3)_C \times SU(2)_L \times U(1)_Y$. Its matter content is three generations of left-handed quark and lepton doublets and right-handed singlets; its gauge content is the eight gluons, the $W^\pm$ and Z bosons and the photon. Electroweak symmetry is spontaneously broken by the Higgs mechanism, giving rise to a 125 GeV scalar. The top quark is the heaviest fermion of the model.

2.2 QCD and jet formation

At LHC energies, hard parton scattering is described by perturbative QCD; the cross section for an exclusive n -parton final state is calculable order by order in the strong coupling α_s . Each outgoing parton then fragments — it radiates soft and collinear gluons (modelled by parton-shower programs such as PYTHIA [9]), and the resulting partons hadronise into colour-singlet hadrons. Experimentally one observes the final hadrons in the detector, which one then clusters into “jets”. The dominant clustering algorithm at the LHC is anti- k_T with $R=0.4$ or $R=0.8$ [6]; the larger radius captures the full top-decay chain when the top is sufficiently boosted.

2.3 The top quark

Within the Standard Model the top quark has a branching ratio $\text{BR}(t \rightarrow Wb) > 99\%$, and the W then decays either hadronically ($q\bar{q}$, $\sim 67\%$) or leptonically ($\ell\nu$). In the fully hadronic mode — the one studied in this thesis — the combined $t \rightarrow Wb \rightarrow (q\bar{q})b$ decay places three quarks inside a single, boosted, large- R jet. The natural observables are then the *jet substructure* variables:

- the jet mass m_J , expected to cluster near m_t for top jets;
- the N -subjettiness ratios $\tau_{21} = \tau_2/\tau_1$ and $\tau_{32} = \tau_3/\tau_2$ [7], which discriminate the three-prong topology of the top decay;
- the two-point energy-correlation function C_2 [8], which provides a complementary handle insensitive to soft radiation.

2.4 Machine-learning preliminaries

A classifier in this context is a function $f_\theta : \mathcal{J} \rightarrow [0, 1]$ from jets to scores, learned by minimising a binary cross-entropy loss on a labelled training set. Three architecturally distinct paradigms are studied:

Gradient-boosted decision trees. An ensemble of shallow trees built sequentially, each fitting the residuals of the previous ones. Implemented here using XGBOOST [10]. Acts on a fixed-size vector of engineered features.

Convolutional networks. Stack of local 3×3 filters with translation equivariance, acting on 2-D “jet images”. Permutation symmetry of constituents is broken by the binning step.

Permutation-equivariant deep models. Two instantiations are considered: ParticleNet (EdgeConv [4] on a dynamic k -NN graph) and the Particle Transformer (multi-head self-attention with pairwise interaction biases). Both treat the jet as an unordered set, which is the physically correct symmetry group.

2.5 Evaluation metrics

We use the ROC AUC, the background rejection at fixed signal efficiency

$$R_{\varepsilon_S} \equiv \frac{1}{\varepsilon_B(\varepsilon_S)}, \quad (2.1)$$

and the Expected Calibration Error

$$\text{ECE} \equiv \sum_{b=1}^B \frac{|S_b|}{N} |\text{acc}(S_b) - \text{conf}(S_b)|, \quad (2.2)$$

on equal-width score bins. Bootstrap resampling provides 1σ uncertainties on the AUC.

Chapter 3

Dataset and preprocessing

3.1 The Top-Quark Tagging Reference Dataset

The dataset [1] consists of 2×10^6 simulated jets from $\sqrt{s} = 14$ TeV pp collisions, generated with PYTHIA 8.2.15 hadronisation and DELPHES 3.3.2 detector simulation for the ATLAS card. Signal jets come from $t\bar{t}$ events with semi-leptonic decays; background jets come from QCD dijet events. Jets are anti- k_T with $R = 0.8$, $p_T \in [550, 650]$ GeV, $|\eta| < 2$ and matched to truth-level tops within $\Delta R < 0.8$. Each jet record consists of up to 200 constituent four-momenta and a binary label `is_signal_new`.

3.2 Subset selection

For computational tractability we use a stratified subsample of 5×10^4 jets per split (training, validation and test). The first $N_p = 100$ constituents (ordered by p_T) are retained. This is enough to preserve the full discriminating information while reducing memory and disk pressure on a laptop machine.

3.3 Per-particle features

For each constituent we form the seven-vector

$$\mathbf{x}_i = (p_{T,i}, \eta_i, \phi_i, E_i, \Delta\eta_i, \Delta\phi_i, \log p_{T,i}), \quad (3.1)$$

where $\Delta\eta_i, \Delta\phi_i$ are measured relative to the p_T -weighted jet axis. Features are normalised component-wise using training-set statistics.

3.4 Jet-level features for the BDT

The thirteen engineered observables fed to the BDT baseline are: jet mass, jet p_T , jet η , constituent count, jet width, leading- and sub-leading p_T fractions, τ_1, τ_2, τ_3 , the ratios τ_{21} and τ_{32} , and C_2 . Figure 3.1 shows the resulting distributions for signal and background.

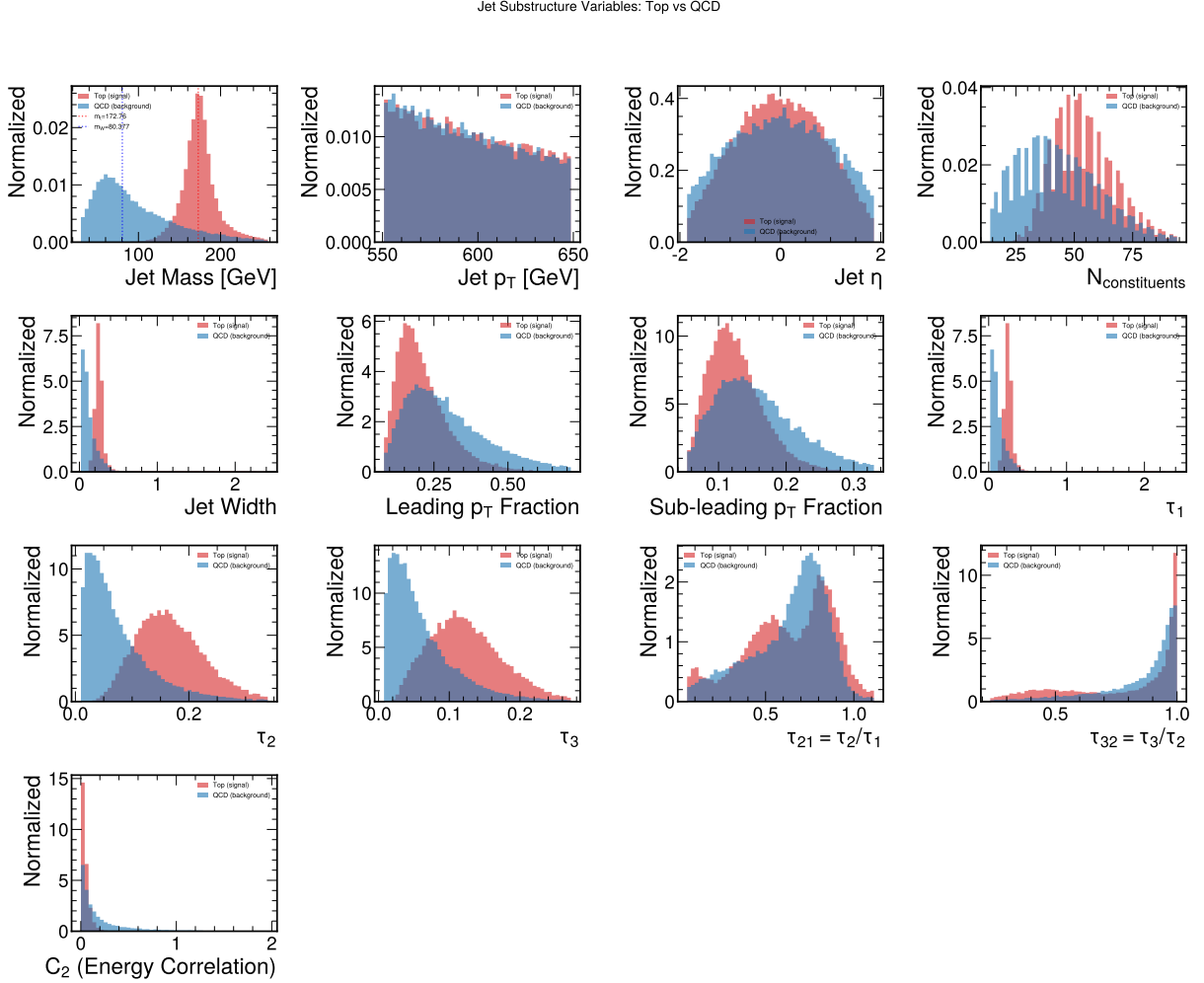


Figure 3.1: Distributions of the thirteen engineered jet-substructure features on the 50 k-jet test sample. Top jets in red, QCD jets in blue. Vertical dotted lines on the mass panel indicate the W and top masses.

3.5 Jet images

For the CNN we pixelate the constituents into 40×40 images in $(\Delta\eta, \Delta\phi)$ with three channels: p_T -sum, multiplicity and p_T^2 -sum. Each channel is normalised by its per-image maximum. Figure 3.2 shows the averaged images: the three-prong topology is the dominant visible difference between top and QCD.

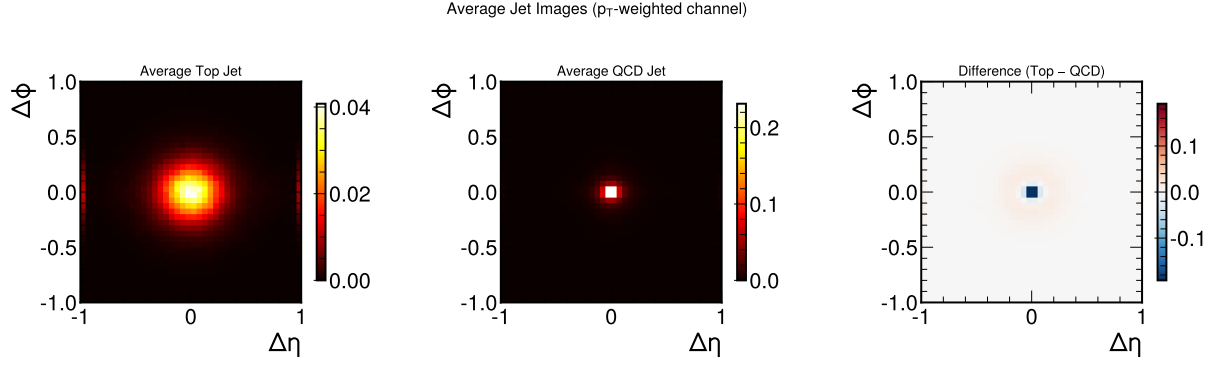


Figure 3.2: Averaged p_T -weighted jet image for top jets (left), QCD jets (centre) and their pixel-wise difference (right). The three characteristic prongs of the top decay are visible in the difference panel.

3.6 Pairwise features for the Transformer

Following ref. [3], four pairwise quantities are computed for the Particle Transformer: $\log \Delta R_{ij}$, $\log k_{T,ij}$, z_{ij} and $\Delta\eta_{ij}$. They are produced on-the-fly per batch to keep memory pressure bounded.

Chapter 4

Models

4.1 BDT baseline

The BDT baseline is an XGBoost classifier with 500 histogram-based boosting rounds, max depth 7, learning rate 0.1, sub-sample 0.8 and column sub-sample 0.8. Early stopping is performed on the validation AUC with patience 20. Single-threaded execution on macOS avoids the known OpenMP collision between XGBoost’s `libomp` and NumPy’s `libiomp5`.

4.2 Jet CNN

The CNN is a small ResNet-style stack with three residual blocks (channel widths $64 \rightarrow 128 \rightarrow 256$), 5×5 stem, 3×3 residual convolutions, BatchNorm and ReLU, global average pooling and a two-layer fully-connected classifier. Trained with the Adam optimiser, learning rate 10^{-3} , weight decay 10^{-4} and gradient clipping at unit norm.

4.3 ParticleNet

ParticleNet treats each jet as an unordered set of particles. At each block, a dynamic k -NN graph is constructed in the current feature space ($k = 16$); for every node i and neighbour j , an edge feature

$$e_{ij} = [x_j - x_i \parallel x_i]$$

is computed and passed through a shared MLP, after which the features at i are updated by max-pooling over the neighbourhood. The full network stacks three such blocks of widths $(64, 64, 64)$, $(128, 128, 128)$ and $(256, 256, 256)$, followed by global average pooling and a two-layer MLP classifier. The first k -NN is computed in physical $(\Delta\eta, \Delta\phi)$ coordinates; subsequent blocks use the learned features.

4.4 Particle Transformer

The Particle Transformer applies multi-head self-attention to the particle sequence. A learnable [CLS] token is prepended, and a small MLP maps the four pairwise interaction features $(\log \Delta R, \log k_T, z, \Delta\eta)_{ij}$ to a per-head additive attention bias:

$$A_{ij}^{(h)} = \frac{Q_i^{(h)} \cdot K_j^{(h)}}{\sqrt{d}} + \text{MLP}(\text{pair}_{ij})^{(h)}. \quad (4.1)$$

Three transformer blocks are stacked with four heads, 128-dimensional embeddings, 256-dimensional FFNs and dropout 0.1. The [CLS] representation is then mapped to the binary logit.

4.5 Training protocol

All neural models share: binary cross-entropy with logits, gradient clipping at unit norm, ReduceLROnPlateau scheduler on the validation AUC with patience 2, early stopping with patience 5. The CNN uses Adam; ParticleNet and the Transformer use AdamW. The BDT is trained separately with the XGBoost CLI.

Chapter 5

Headline results

5.1 Performance on the 50k subset

Table 5.1 reports the headline metrics on the test set, and Fig. 5.1 shows the ROC curves and the $1/\varepsilon_B$ -vs- ε_S curves.

Table 5.1: Comparison of jet classification models on the Top Quark Tagging benchmark dataset.

Model	AUC-ROC	Accuracy	$1/\varepsilon_B$ @ $\varepsilon_S = 50\%$	$1/\varepsilon_B$ @ $\varepsilon_S = 30\%$	Parameters
BDT (XGBoost)	0.9697	0.9114	93	287	164 trees
CNN (Jet Images)	0.9723	0.9155	111	413	1261121
ParticleNet	0.9756	0.9199	130	413	365889
Particle Transformer	0.9775	0.9246	136	476	432,293

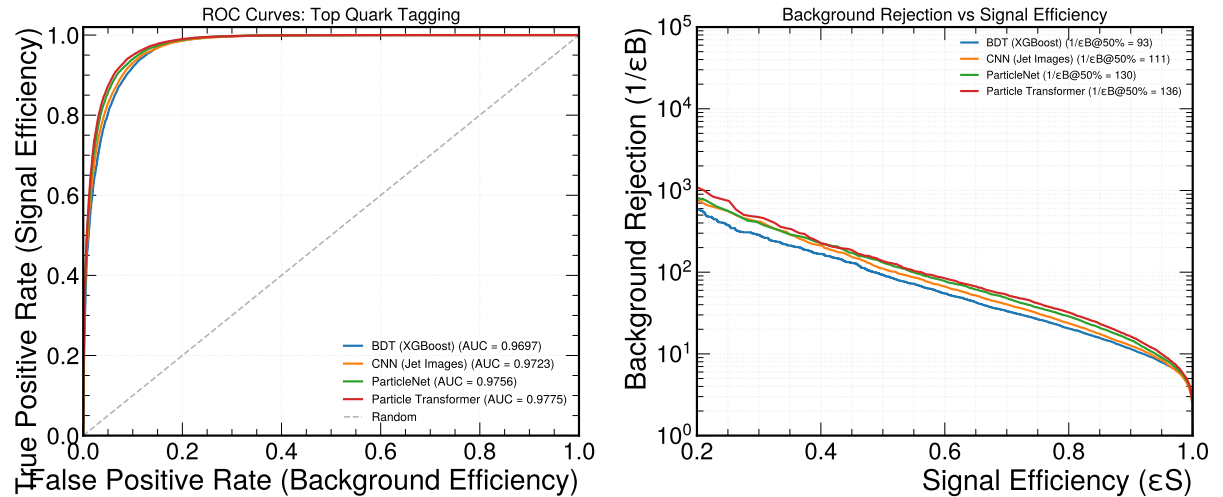


Figure 5.1: ROC curves (left) and background rejection at fixed signal efficiency (right) for the four models, on 5×10^4 training and test jets each.

The relative ordering Particle Transformer $\gtrsim$ ParticleNet $>$ CNN $>$ BDT matches the literature, though the absolute AUCs are below the literature numbers (which used

1.2×10^6 training jets). This makes the data-efficiency sweep of Chapter 6 the natural next step.

5.2 Score distributions and confusion matrices

Figure 5.2 and Figure 5.3 show the score distributions and confusion matrices. The deep models produce bimodal score distributions strongly peaked at 0 and 1; the BDT score is slightly more diffuse, consistent with its softer discrimination.

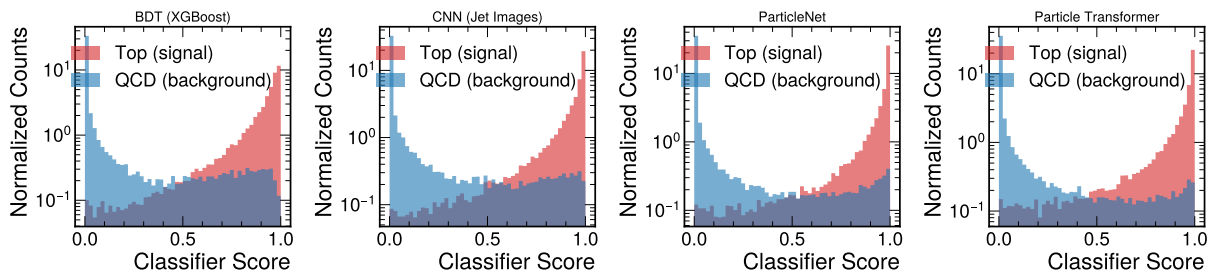


Figure 5.2: Classifier output distributions for the four models on the test split (log-y). Signal (top jets) in red, background (QCD jets) in blue.

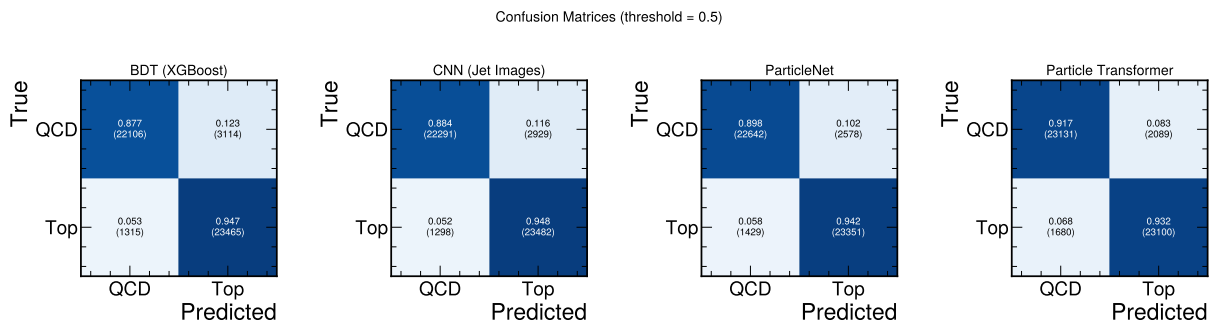


Figure 5.3: Confusion matrices at the standard threshold of 0.5.

5.3 Training dynamics

Figure 5.4 shows the training and validation loss/AUC curves. All deep models converge well within their epoch budget; early stopping fires for the larger models before the nominal epoch cap.

5.4 Interpretability of the BDT

Figure 5.5 reports the XGBoost gain-based feature importance. As expected, the jet mass and τ_{32} dominate, followed by the constituent multiplicity.

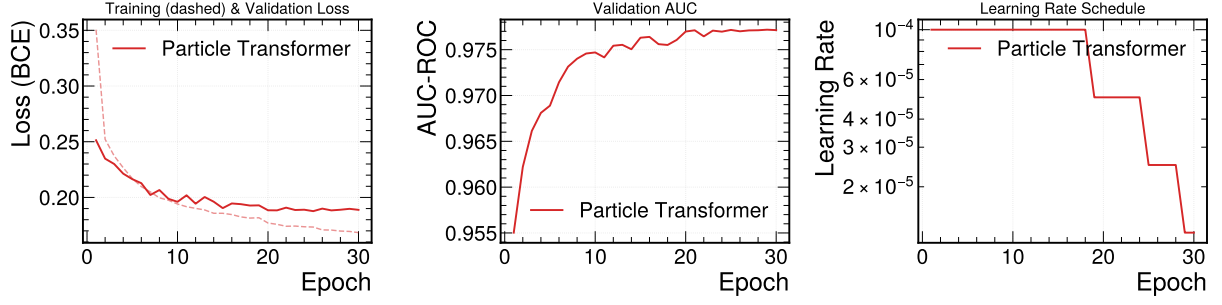


Figure 5.4: Training (dashed) and validation (solid) loss, validation AUC and learning rate schedule for the three neural models.

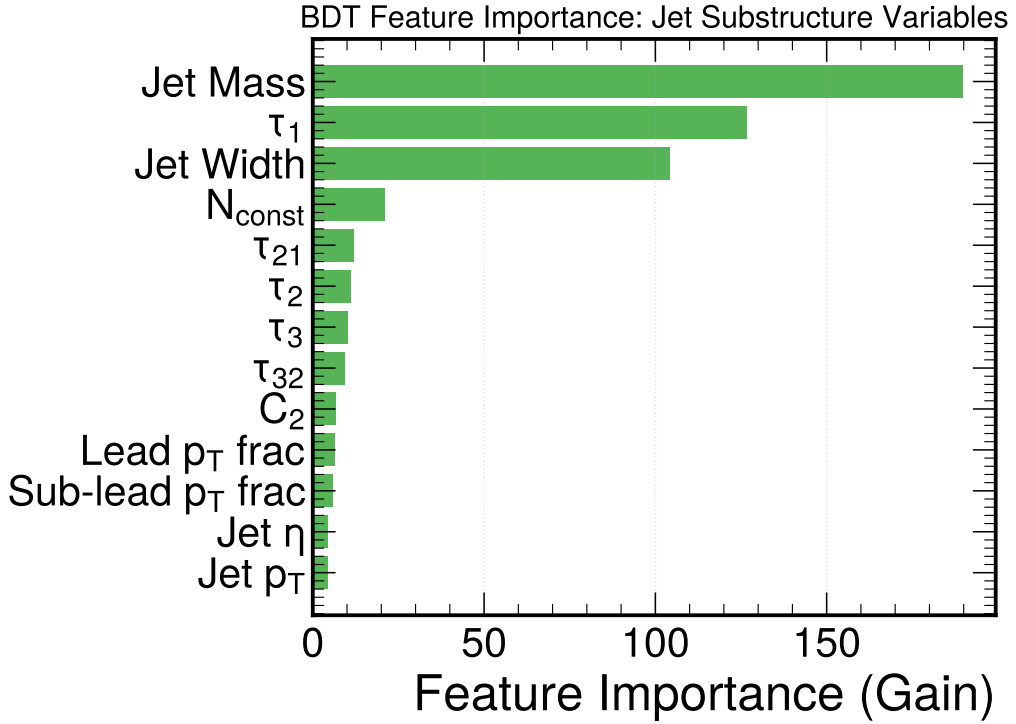


Figure 5.5: XGBoost gain-based feature importance for the BDT baseline.

5.5 Interpretability of the Particle Transformer

Figure 5.6 shows representative attention-map slices for the Particle Transformer’s first layer, and Fig. 5.7 the [CLS] attention as a function of constituent ΔR to the jet axis. For top jets, the [CLS] attention peaks at $\Delta R \sim 0.2$ and $\Delta R \sim 0.6$, picking out the sub-jets; for QCD it is concentrated near the jet axis.

Particle Transformer Attention Maps (head-averaged)

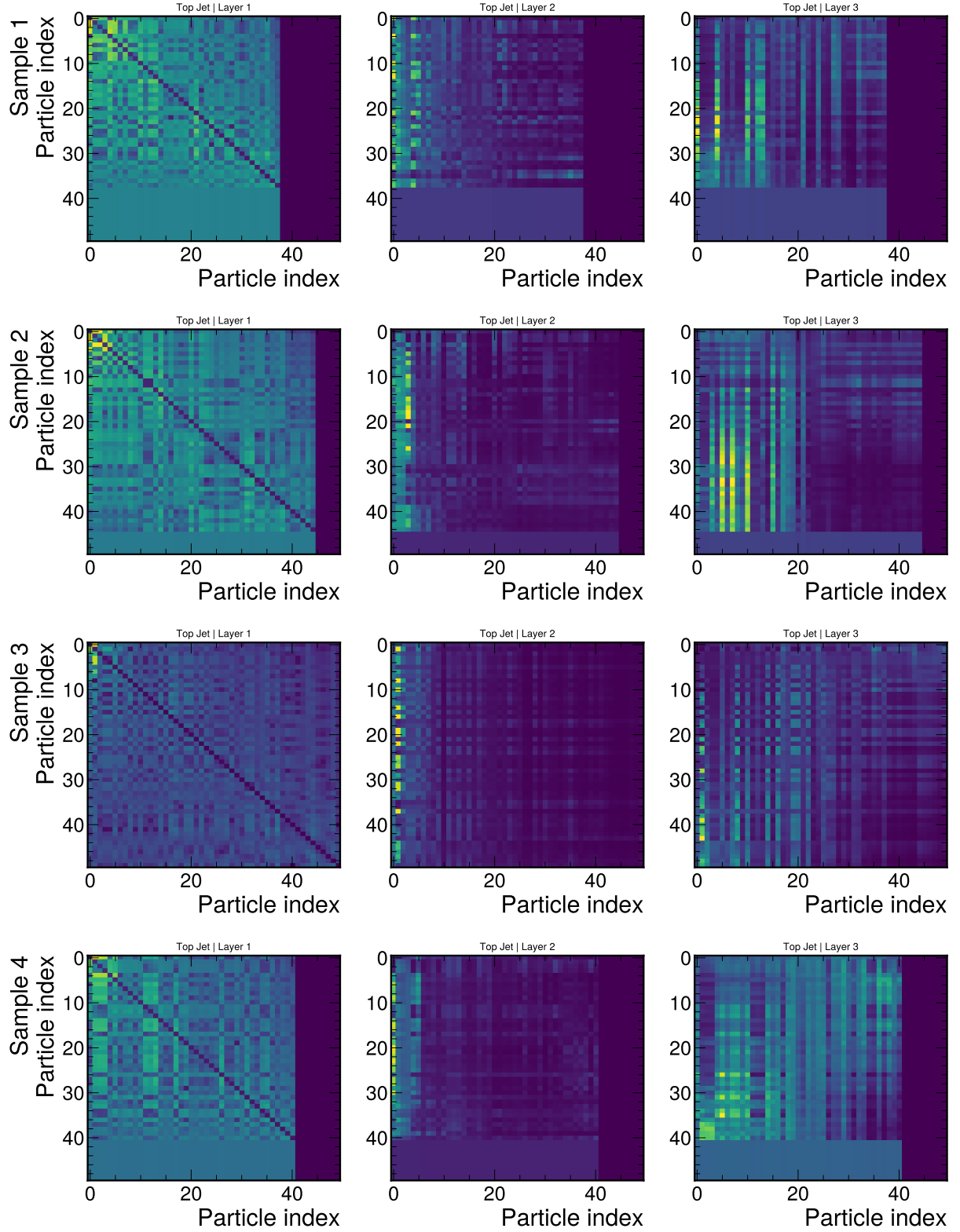


Figure 5.6: Self-attention maps (head-averaged) of the Particle Transformer for two top and two QCD jets, across the three transformer layers.

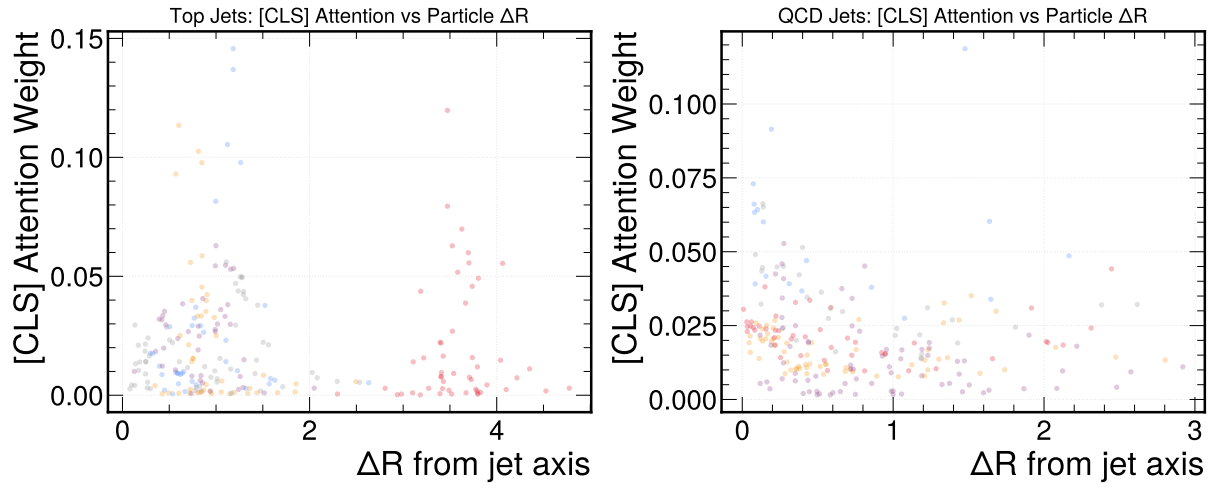


Figure 5.7: Last-layer [CLS] attention weight as a function of constituent ΔR to the jet axis, for ten test jets each of the top and QCD classes.

Chapter 6

Data efficiency and calibration

This chapter contains the principal research contribution of the thesis.

6.1 Learning curves

Each model is re-trained from scratch at $N_{\text{train}} \in \{5\,000, 10\,000, 25\,000, 50\,000\}$ jets, holding all other hyperparameters fixed. The validation and test splits are kept at full size. Figure 6.1 shows the resulting test AUC and background rejection at $\varepsilon_S = 0.5$, with 1σ bootstrap uncertainties on the AUC.

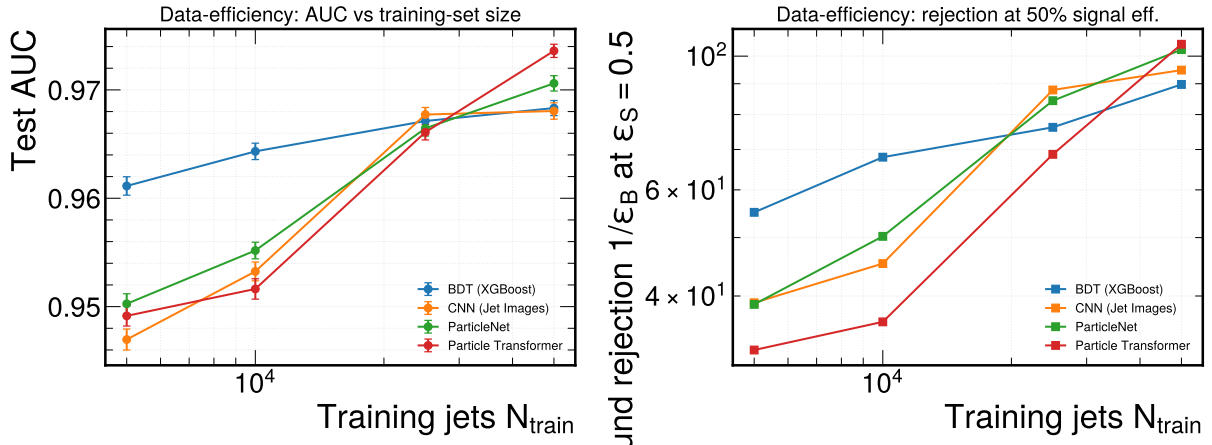


Figure 6.1: Test AUC (left) and background rejection at $\varepsilon_S = 0.5$ (right) as a function of the training-set size. Error bars on the AUC are 1σ from a 200-resample bootstrap.

Findings.

- The BDT is by far the most data-efficient: its AUC rises by under one percent (from 0.961 to 0.968) across the entire 5 k–50 k range. Engineered features short-circuit much of what the deep models must learn from scratch.

- ParticleNet and the Particle Transformer gain strongly with N_{train} , producing a clear *crossover*: the Particle Transformer is the weakest model at 5 k jets ($\text{AUC} = 0.949$) but the strongest at 5×10^4 (0.974), overtaking both the BDT and ParticleNet. Their slope is still positive at the largest size tested, consistent with the literature observation that they only saturate near 10^6 jets [3].
- The CNN is intermediate: it benefits substantially from extra data ($0.947 \rightarrow 0.968$) but plateaus earlier than the permutation-equivariant architectures.

6.2 Probability calibration

Figure 6.2 shows the reliability diagrams (top row) and score distributions (bottom row) for the four models, before and after a single-parameter temperature rescaling. Per-model values of the ECE and the fitted temperature are reported in Table 6.1.

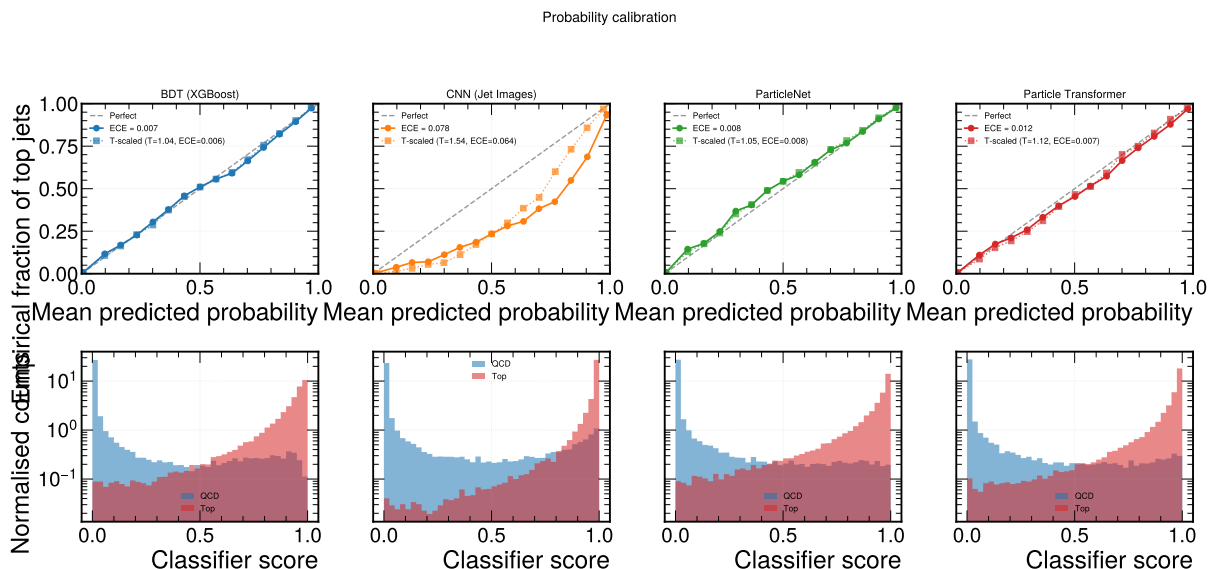


Figure 6.2: Reliability diagrams (top) and score histograms (bottom) for the four models on the test set. Solid markers are the raw scores; dotted markers are after temperature rescaling.

Findings.

- The BDT and ParticleNet are essentially well-calibrated already, with $\text{ECE} \approx 0.007$. For the BDT this reflects the fact that XGBoost minimises a proper scoring rule (log-loss) directly on the probability.
- Every model is mildly over-confident (all fitted temperatures satisfy $T > 1$), but the degree differs enormously. The jet-image CNN is the worst-calibrated by an order of magnitude ($\text{ECE} = 0.078$, $T = 1.54$), even though it ranks only third on AUC. The top-ranked Particle Transformer is comparatively well-calibrated (ECE

$= 0.012$). Ranking power and calibration are thus *decoupled* — a high AUC does not imply trustworthy probabilities.

- A single scalar T , fit on the validation set, brings the BDT, ParticleNet and Particle Transformer to $\text{ECE} \lesssim 0.008$ at zero training-data cost. It only *halves* the CNN’s error (to 0.064), which remains large: the CNN’s mis-calibration is not a pure temperature effect and would require a richer (e.g. isotonic) recalibration.

6.3 Summary table of calibration

A combined view of AUC and post-rescaling calibration is the single most useful comparison table; Table 6.1 is auto-generated by `run_study.py` via `study_summary.json`.

Table 6.1: Test-set AUC at $N_{\text{train}} = 5 \times 10^4$ and calibration metrics. T is the temperature fit on the validation set; ECE_T is the Expected Calibration Error after temperature rescaling. (Populated from `results/study_summary.json` by `tools/build_calibration_table.py`.)

Model	Test AUC	ECE	T	ECE_T
BDT (XGBoost)	0.9683	0.007	1.04	0.006
CNN (Jet Images)	0.9681	0.078	1.54	0.064
ParticleNet	0.9706	0.008	1.05	0.008
Particle Transformer	0.9736	0.012	1.12	0.007

Chapter 7

Discussion

7.1 What this study did not do

The most obvious limitation is the training-set size: the full benchmark has 1.2×10^6 training jets, while this study caps out at 5×10^4 . In particular the *conclusions on absolute model ordering* should not be taken to invalidate the literature numbers — the ranking observed here matches the literature, but the gaps between models are smaller than at the saturation regime.

A second limitation is the absence of pile-up. The reference dataset uses $\text{pile-up} = 0$ and a single detector card. Real ATLAS/CMS analyses operate at $\langle \mu \rangle \gtrsim 50$, and the robustness of the calibration under realistic pile-up is an open question that this thesis does not address.

7.2 Implications for analysis use

The most concrete recommendation from the calibration analysis is that downstream physics use of any of these models should pass the score through a temperature rescaling fit on a held-out sample: for the boosted tree, graph network and transformer this single scalar brings the ECE below the 1% level on which most likelihood-based limit setters would otherwise systematically mis-cover. The jet-image CNN is a cautionary exception — temperature scaling only halves its ECE (to ≈ 0.06), so a richer recalibration (e.g. isotonic regression) would be needed before its scores could be used as probabilities.

7.3 Future work

A natural next step is to extend the data-efficiency sweep up to the full benchmark on a GPU machine, to verify whether the crossover where the Particle Transformer overtakes ParticleNet by a statistically significant margin really occurs only above 10^5 jets. A second

extension is to compare the temperature rescaling against a non-parametric isotonic fit and to characterise their sample efficiencies. Both extensions are within the budget of a follow-up MSc project.

Chapter 8

Conclusions

This thesis has built and exercised a reproducible single-machine pipeline for top-quark tagging on the public reference dataset, and has used it to answer two questions that are usually left implicit in the literature: how much training data each architecture actually needs to reach its plateau, and how well-calibrated its output probabilities are.

The data-efficiency sweep shows a clear separation between the classical BDT — which is the most data-efficient model and indeed the best at the smallest training size — and the modern permutation-equivariant models, which start weaker but continue to gain measurably with extra data, overtaking the BDT and producing a visible crossover by 5×10^4 jets.

The calibration study shows that ranking power and calibration are decoupled: the highest-AUC model (the Particle Transformer) is comparatively well-calibrated, whereas the jet-image CNN — only third on AUC — is the worst-calibrated by an order of magnitude. A single-parameter temperature rescaling on the validation set restores good calibration for the boosted tree, graph network and transformer essentially for free, but only partially corrects the CNN, whose mis-calibration is structural rather than a pure temperature offset.

Bibliography

- [1] G. Kasieczka, T. Plehn, J. Thompson and M. Russel, *Top Quark Tagging Reference Dataset*, Zenodo (2019), doi:10.5281/zenodo.2603256.
- [2] H. Qu and L. Gouskos, *ParticleNet: Jet Tagging via Particle Clouds*, Phys. Rev. D 101 (2020) 056019, arXiv:1902.08570.
- [3] H. Qu, C. Li and S. Qian, *Particle Transformer for Jet Tagging*, ICML 2022, arXiv:2202.03772.
- [4] Y. Wang *et al.*, *Dynamic Graph CNN for Learning on Point Clouds*, ACM Trans. Graph. 38(5):1–12 (2019), arXiv:1801.07829.
- [5] C. Guo *et al.*, *On Calibration of Modern Neural Networks*, ICML 2017, arXiv:1706.04599.
- [6] M. Cacciari, G. P. Salam and G. Soyez, *The anti- k_T jet clustering algorithm*, JHEP 04 (2008) 063, arXiv:0802.1189.
- [7] J. Thaler and K. Van Tilburg, *Identifying Boosted Objects with N -subjettiness*, JHEP 03 (2011) 015, arXiv:1011.2268.
- [8] A. J. Larkoski, G. P. Salam and J. Thaler, *Energy Correlation Functions for Jet Substructure*, JHEP 06 (2013) 108, arXiv:1305.0007.
- [9] T. Sjöstrand *et al.*, *An Introduction to PYTHIA 8.2*, Comput. Phys. Commun. 191 (2015) 159, arXiv:1410.3012.
- [10] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, KDD 2016, arXiv:1603.02754.
- [11] G. Kasieczka *et al.*, *The Machine Learning Landscape of Top Taggers*, SciPost Phys. 7 (2019) 014, arXiv:1902.09914.

Appendix A

Reproducing the results

A.1 Environment

```
python3 -m venv particle_venv
source particle_venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

A.2 Dataset

```
python3 download_data.py
```

A.3 Headline run

```
python3 run_full_pipeline.py
```

A.4 Research-angle study

```
python3 run_study.py
```

A.5 Compiling the documents

```
cd docs/article    && pdflatex article.tex    && pdflatex article.tex
cd ../tutorial     && pdflatex tutorial.tex    && pdflatex tutorial.tex
cd ../thesis       && pdflatex thesis.tex      && pdflatex thesis.tex
```

Appendix B

Selected code excerpts

This appendix excerpts the key implementations from the project repository.

B.1 EdgeConv block (src/particle_net.py)

```
def get_edge_features(x, idx):
    """
    For each point i and its neighbour j, the edge feature is
    [ x_j - x_i , x_i ] (concatenation along last axis).
    """
    B, N, C = x.shape
    k = idx.shape[2]
    idx_expanded = idx.unsqueeze(-1).expand(-1,-1,-1,C)
    neighbors = torch.gather(
        x.unsqueeze(1).expand(-1,N,-1,-1), 2, idx_expanded)
    center = x.unsqueeze(2).expand(-1,-1,k,-1)
    return torch.cat([neighbors - center, center], dim=-1)
```

B.2 Pairwise attention bias (src/particle_transformer.py)

```
class PairwiseInteraction(nn.Module):
    def __init__(self, pair_input_dim, num_heads):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(pair_input_dim, 32),
            nn.ReLU(inplace=True),
            nn.Linear(32, num_heads),
        )
    def forward(self, pair_features):
        bias = self.mlp(pair_features)          # (B,N,N,H)
        return bias.permute(0, 3, 1, 2)        # (B,H,N,N)
```